

Cross-Site Request Forgery (CSRF) Attack Lab

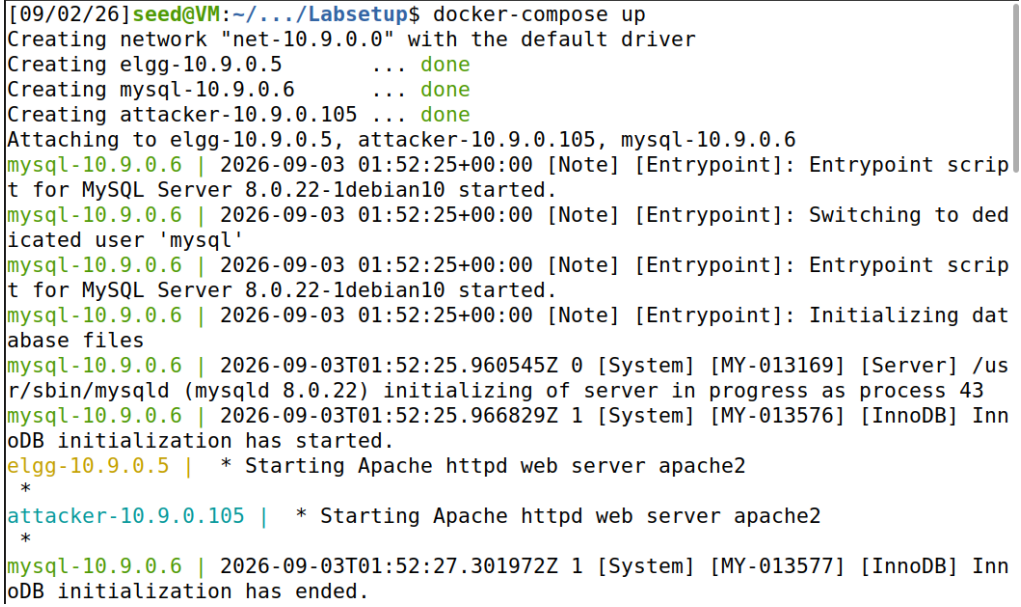
姓名：王子卓

学号：57123242

实验报告需求：按实验指导书按步执行，实验报告记录执行的命令，对执行结果截屏，并对指导书中提出的问题进行回答和截屏展示。

1、实验环境设置

1.1 启动 Docker，并进入 docker 环境, 准备实验

任务：启动 docker 并进入实验环境，检查容器、DNS 和数据库设置
命令： docker-compose build docker-compose up
截图： 
图 1：创建 docker 容器
dockps
截图：

<pre>[09/02/26]seed@VM:~/.../Labsetup\$ dockps 1db6635f6e6a mysql-10.9.0.6 d553b162da44 attacker-10.9.0.105 546fa9518843 elgg-10.9.0.5</pre>	
图 1:	
<pre>sudo nano /etc/hosts 10.9.0.5 www.seed-server.com 10.9.0.5 www.example32.com 10.9.0.105 www.attacker32.com</pre>	
<pre>cat /etc/hosts grep seed-server DNS 和数据库检查</pre>	
截图: <pre>[09/02/26]seed@VM:~/.../Labsetup\$ cat /etc/hosts grep seed-server [09/02/26]seed@VM:~/.../Labsetup\$ sudo nano /etc/hosts [09/02/26]seed@VM:~/.../Labsetup\$ cat /etc/hosts grep seed-server 10.9.0.5 www.seed-server.com [09/02/26]seed@VM:~/.../Labsetup\$ █</pre> <pre># For CSRF 10.9.0.5 www.seed-server.com 10.9.0.5 www.example32.com 10.9.0.105 www.attacker32.com█</pre> <pre># For Shellshock Lab 10.9.0.80 www.seedlab-shellshock.com</pre>	
图 1: 修改/etc/hosts 文件	

2、实验步骤

2.1 Task 1: Observing HTTP Request.

任务: 根据任务书 5.1 指南学习使用 Firefox 插件观察 HTTP 请求头部。
位置: SEED
命令 : 用户名: alice 密码: seedalice
截图

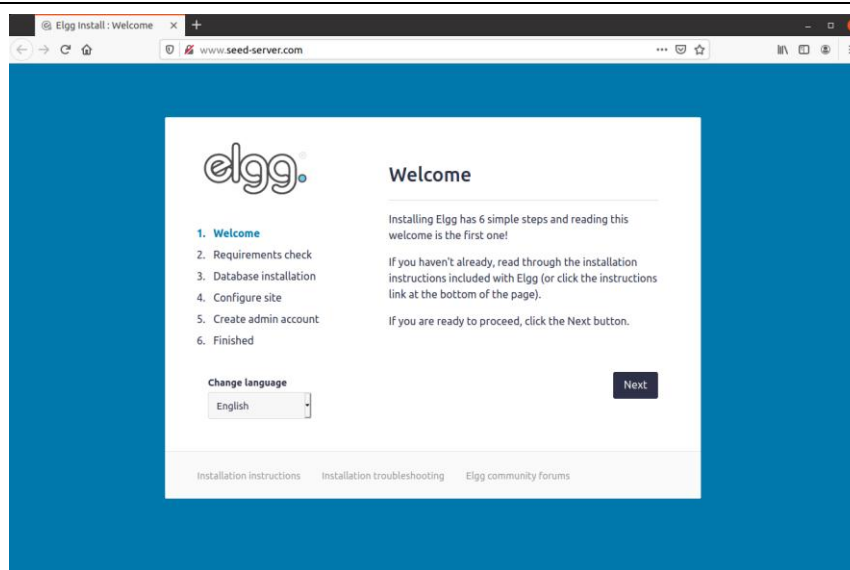


图 1：完成 install 的六个步骤

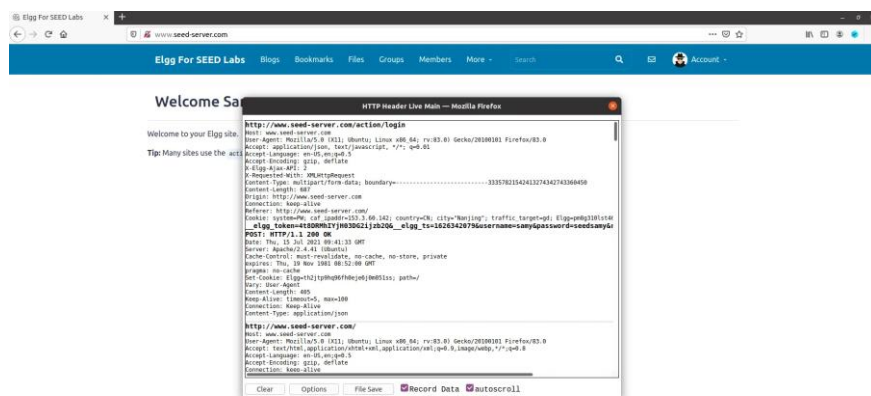


图 2：登录并且通过 header live 观察报头

结果说明：

朋友们正常情况下是没有这个下载环节的，可以直接用密码登录进去。笔者的电脑不知道除了什么问题，一进入这个网页就 fatal error 报错或者要从头开始下载，卡了很久。如果也遇到相同的问题，可以考虑

关于：报错及解决方案，进入网站以后是下载界面，不是登录界面，只好手动下载数据库

Settings file

Error

A settings file exists in the installation directory, but the web server cannot read it. You can delete the file or change the read permissions on it.

docksh 546

cd /var/www/elgg/engine

ls -l /var/www/elgg/elgg-config/settings.php

chmod 644 /var/www/elgg/elgg-config/settings.php

```
cat /var/www/elgg/elgg-config/settings.php | grep dataroot
cat /var/www/elgg/elgg-config/settings.php | grep -E
"dbhost|dbuser|dbpass|dbname"
```

2.2 Task 2: CSRF Attack using GET Request

任务：按任务书要求，描述如何根据实验模版网页构造攻击网页，让 Alice 添加 samy 为好友。

环境：SEED

命令

截图（可以有多组）

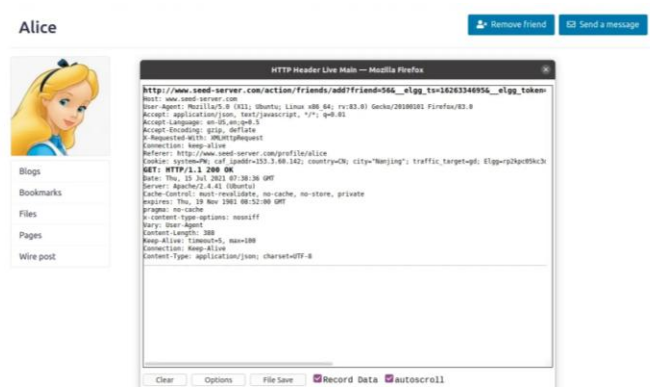


图 1：点进 alice 的主页，添加她为好友，看到加好友的请求语句是 GET

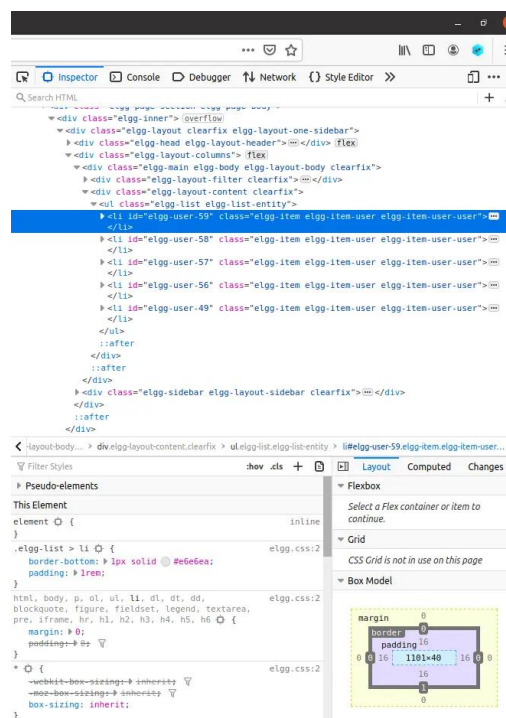


图 2：在 members 页面 F12 查看列表，找到明文储存的用户 id

```
1 <html>
2 <body>
3   <h1>欢迎来到我的精彩小站！</h1>
4   
6 </body>
7 </html>
```

图 3：攻击页面代码

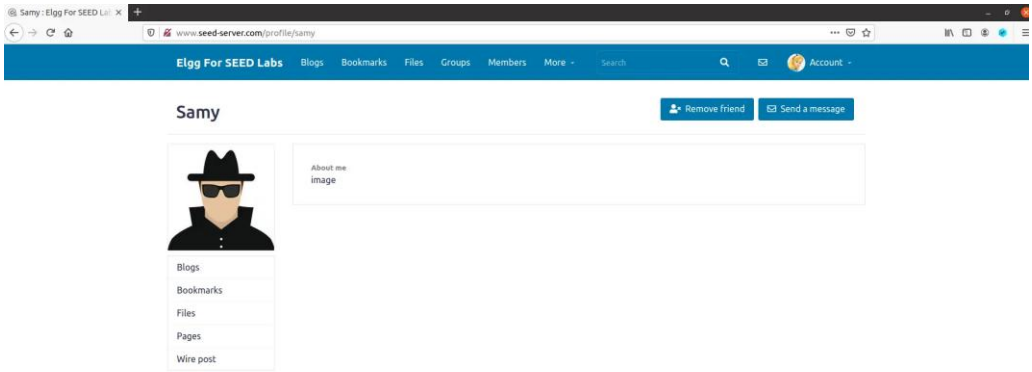


图 4：登录 Alice 的帐号，会发现 Samy 已经成功添加进 Alice 的好友列表了

结果说明：

在加载图片时，浏览器会自动发一个 **GET 请求**。Samy 把这个图片的 src 属性，写成添加好友的 URL（比如 <http://www.seed-server.com/action/friends/add?friend=59>，这里的 59 是我们的 uid）。Alice 只要点开这个图片页面，浏览器就会自动发出加好友请求，Samy 直接成为 Alice 好友，无需 Alice 点击任何按钮。

2.3 Task 3: CSRF Attack using POST Request

任务：按任务书要求，描述如何根据实验模版网页构造攻击网页，让 Alice 修改 profile 添加一句“Samy is my Hero”。
环境：SEED
命令
截图（可以有多组）

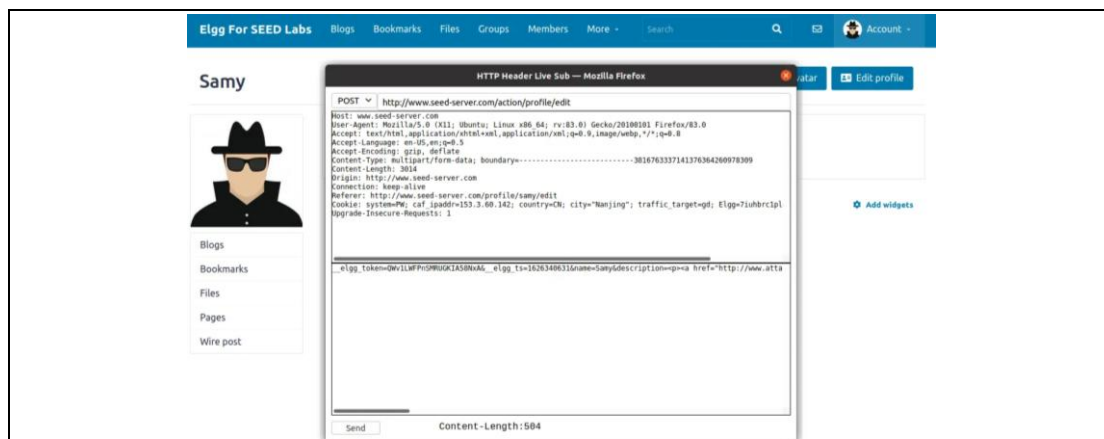


图 1：通过修改个人 file，发现使用 POST 请求，URL 是 <http://www.seed-server.com/action/profile/edit>

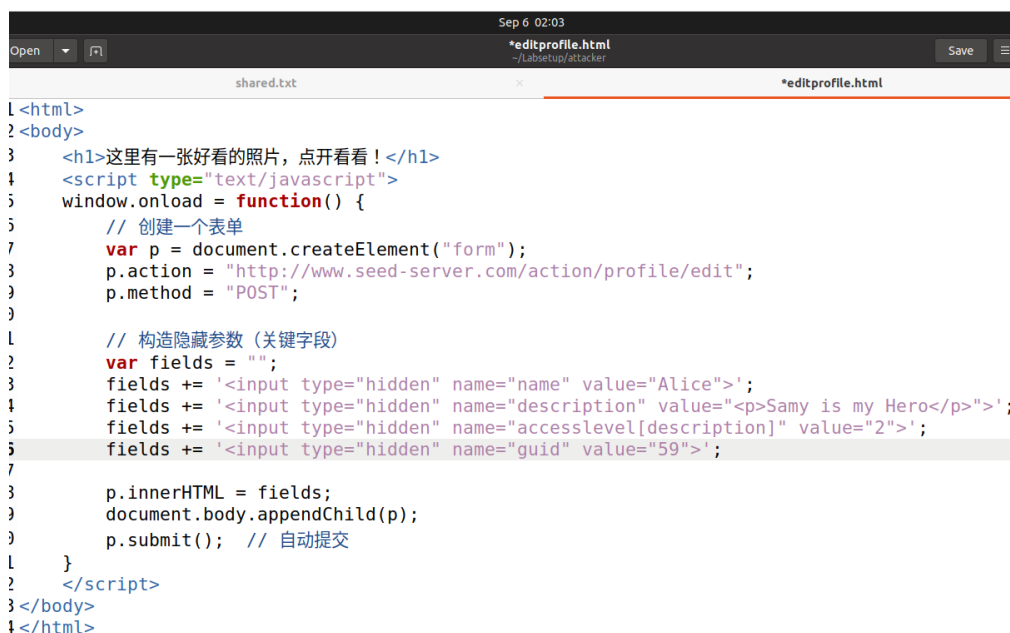


图 2：编辑 Editfile.html 文件

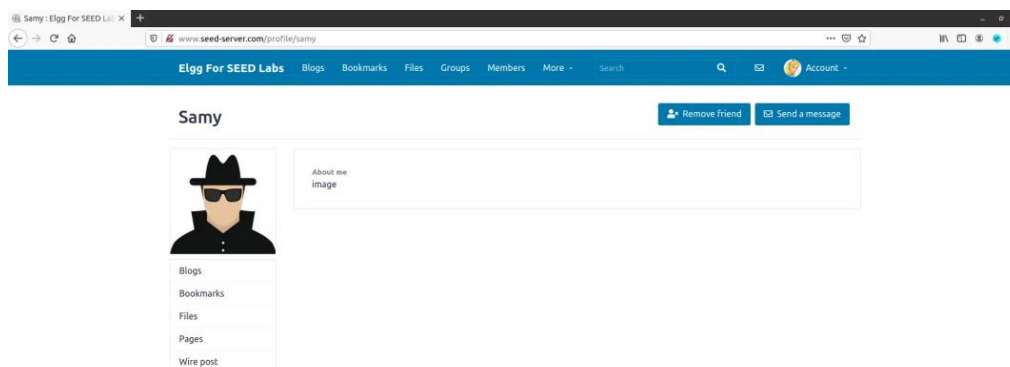


图 3：制作一个诱导 alice 点击的链接放到 samy 主页

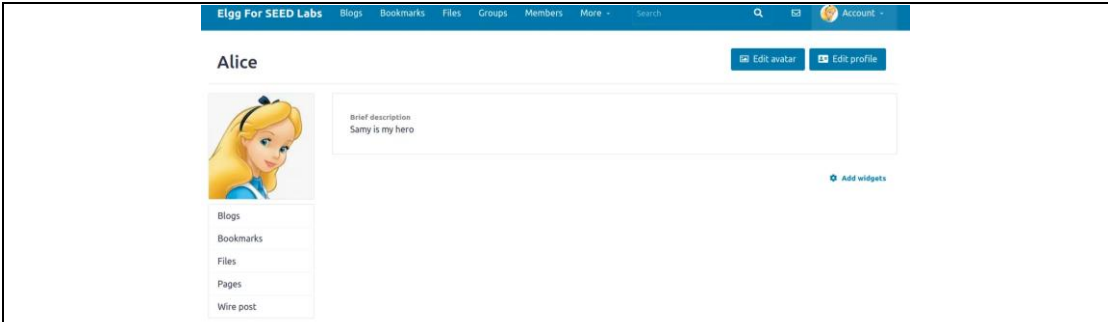


图 4: 登录 alice 的帐号点击 image, 可以看到 Alice 的 profile 已经修改了

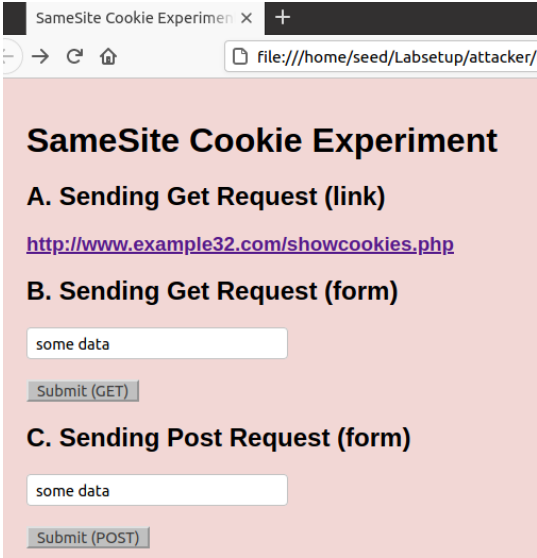
问题 1, 请给出获取 Alice 的 userid 的方法
攻击者虽然不知道 Alice 的密码, 但 Alice 的个人资料页面是公开的。Boby 可以直接在浏览器里访问 <code>http://www.seed-server.com/profile/alice</code> , 然后查看网页源代码, 或者直接看地址栏中头像链接的 guid 数字, 从而提前获取 Alice 的 ID。如果需要, 也可以查看 Alice 头像的 src 属性, 里面通常包含 guid 信息。
问题 2, 如果想获取任意访问用户的 userid, 该如何实现
攻击者不需要提前知道访客的 guid。因为在 Elgg 的后端逻辑中, 修改个人资料 的 <code>edit.php</code> 会优先检查请求中是否有 guid 参数; 如果没有提供, 或者提供的值为空, 服务器会自动从当前会话 (Session) 中获取登录用户的 guid。因此攻击者只需要在伪造的表单里不传 guid 参数 (或传空值), 服务器就会修改当前登录的受害者 (即任何点进来的人) 的资料。
结果说明: 我们这次的目标是修改 Alice 的个人简介, 让她显示 “Samy is my Hero”。所以在 这之前我们先修改自己的 profile, 发现使用的是 POST 请求, 参数在消息体里, 没法像 GET 那样直接用。所以需要 JavaScript 动态创建一个表单 (<form>), 并自动提交 (<code>p.submit()</code>)。

2.4 Task 4: Enabling Elgg' s Countermeasure

任务: 按任务书要求, 打开网站程序注释掉的 secret token 功能, 观察实施 CSRF 攻击的效果, 并说明原因
环境: SEED
命令 docksh f27 nano /var/www/elgg/vendor/elgg/elgg/engine/classes/Elgg/Security/Csrf.php
截图 (可以有多组)

图 1：注释掉 return 这一行，然后关掉容器重启，重复 task3 
图 2：攻击失败，浏览器陷入无限重刷，因为服务器拒绝请求并返回重定向，页面不断重发
结果说明： 开启防御后，Elgg 服务器要求每个修改请求必须携带 <code>__elgg_ts</code> 和 <code>__elgg_token</code> 这两个动态令牌。这两个令牌是 Elgg 在自己的页面里生成的，并且放在隐藏域中，只有 Elgg 同源页面能访问。攻击者的网页（attacker32.com）无法读取 seed-server.com 页面里的 DOM 元素，因此拿不到令牌。没有令牌，服务器就拒绝处理，攻击无效。

2.5 Task 5: Experimenting with the SameSite Cookie Method

任务：按任务书要求，设置不同的 SameSite Cookie 属性值，观察实施 CSRF 攻击的效果，并说明原因 环境：SEED
命令
截图（可以有多组） 
图 1：访问网站
结果说明： 1. Normal（普通，跨站也带） 2. Lax（跨站顶级导航时带，但 POST/iframe 不带） 3. Strict（任何跨站请求都不带） 个人见解：

Elgg 可以把用户的 Session Cookie 的属性设置为 SameSite=Lax 或 Strict。这样，当攻击者页面 (attacker32.com) 向 Elgg (seed-server.com) 发出跨站 POST 请求时，浏览器不会携带 Session Cookie。服务器收不到 Session ID，不知道当前用户是谁，因此 CSRF 攻击无法生效（即使请求发过来，服务器也会因未登录而拒绝）。